TP Fil rouge – API REST “Location de motos” (Node.js / Express / MySQL) – DWWM

Mise en situation

Vous intégrez l’équipe backend de **MotoRent**, une petite entreprise de location de motos.
Le site vitrine existe déjà, mais il manque une **API REST** pour :

- gérer le **catalogue** de motos
- gérer les **clients**
- gérer les **réservations**
- sécuriser certaines routes (admin) via **authentification**

Votre mission : développer une **API Node.js / Express** connectée à **MySQL**.

Contraintes obligatoires (techniques)

- **Mode modules ES** : `"type": "module"` dans `package.json`
- Architecture imposée :
  - `config/` : configuration (connexion MySQL)
  - `models/` : requêtes SQL (accès données)
  - `controllers/` : logique métier (validation, règles, statuts HTTP)
  - `routes/` : routes Express (`Router`)
- **MySQL** via `mysql2/promise`
- **Validation + erreurs** (séance 6) :
  - réponses d’erreurs JSON cohérentes
  - middleware d’erreurs global
  - classe `HttpError` (ou équivalent)
- **Authentification** (séance 7) :
  - mots de passe hashés (`bcrypt`)
  - JWT (`jsonwebtoken`)
- Pour l’instant, vous pouvez mettre la config MySQL et la clé JWT **en dur** dans le code (sécurité plus tard).

Livrables attendus

Dans un dossier `api-location-motos/` :

- un projet Node (`package.json`)
- votre code avec l'architecture imposée
- un fichier `README.md` (installation + commandes + endpoints)
- un fichier `sql/schema.sql` (création des tables)
- un fichier `sql/seed.sql` (quelques données de test)

Partie A – Base de données (obligatoire)

A1) Créer le schéma SQL

Créez les tables suivantes :

- `users` : comptes pour se connecter
- `clients` : clients de la boutique
- `motorcycles` : catalogue de motos
- `reservations` : réservations (liées à un client + une moto)

Contraintes minimales

- `users.email` **unique**
- `clients.email` **unique**
- `reservations` référence `clients` et `motorcycles`

Proposition de schéma (à adapter)

Créez un fichier `sql/schema.sql` :

```sql
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL DEFAULT 'admin'
);

CREATE TABLE clients (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  telephone VARCHAR(30) NULL
);

CREATE TABLE motorcycles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  marque VARCHAR(80) NOT NULL,
  modele VARCHAR(80) NOT NULL,
  cylindree INT NOT NULL,
  prix_jour DECIMAL(10,2) NOT NULL,
  disponible TINYINT NOT NULL DEFAULT 1
);

CREATE TABLE reservations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT NOT NULL,
  motorcycle_id INT NOT NULL,
  date_debut DATE NOT NULL,
  date_fin DATE NOT NULL,
  statut VARCHAR(30) NOT NULL DEFAULT 'CONFIRMED',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (client_id) REFERENCES clients(id),
  FOREIGN KEY (motorcycle_id) REFERENCES motorcycles(id)
);
```

A2) Seed (données de test)

Créez un fichier `sql/seed.sql` avec :

- 2 clients
- 4 motos
- 1 ou 2 réservations

Partie B – API Express (obligatoire)

B1) Initialisation

- `npm init -y`
- `npm install express mysql2 bcrypt jsonwebtoken`
- ajouter `"type": "module"`

B2) Architecture de fichiers (obligatoire)

Arborescence attendue (minimum) :

```
api-location-motos/
├─ package.json
├─ app.js
├─ config/
│  └─ db.js
├─ utils/
│  └─ HttpError.js
├─ middlewares/
│  ├─ authMiddleware.js
│  └─ errorMiddleware.js
├─ validators/
│  ├─ clientValidator.js
│  ├─ motorcycleValidator.js
│  └─ reservationValidator.js
├─ models/
│  ├─ userModel.js
│  ├─ clientModel.js
│  ├─ motorcycleModel.js
│  └─ reservationModel.js
├─ controllers/
│  ├─ authController.js
│  ├─ clientController.js
│  ├─ motorcycleController.js
│  └─ reservationController.js
└─ routes/
   ├─ authRoutes.js
   ├─ clientRoutes.js
   ├─ motorcycleRoutes.js
   └─ reservationRoutes.js
```

> Vous pouvez ajouter des fichiers si besoin, mais cette base est obligatoire.

Partie C – Authentification (obligatoire)

C1) Endpoints

Créer :

- `POST /auth/register`

- `POST /auth/login`

Règles minimales

- email valide (contient `@`)
- password min 6 caractères
- password **hashé** en base
- login renvoie un **JWT** si identifiants corrects

C2) Middleware JWT

Créer `authMiddleware` qui :

- lit `Authorization: Bearer <token>`
- vérifie le token
- remplit `req.user`
- sinon renvoie `401`

Partie D – Ressource “motorcycles” (obligatoire)

D1) Endpoints (CRUD)

- `GET /motorcycles` : liste (accessible sans auth)
- `GET /motorcycles/:id` : détail (sans auth)
- `POST /motorcycles` : création (**protégé JWT**)
- `PUT /motorcycles/:id` : mise à jour (**protégé JWT**)
- `DELETE /motorcycles/:id` : suppression (**protégé JWT**)

D2) Validation minimale

Pour POST/PUT :

- `marque` string min 2
- `modele` string min 1
- `cylindree` nombre positif
- `prix_jour` nombre positif
- `disponible` vaut `0` ou `1`

Partie E – Ressource “clients” (obligatoire)

Endpoints (CRUD)

- `GET /clients` (**protégé JWT**)
- `GET /clients/:id` (**protégé JWT**)
- `POST /clients` (**protégé JWT**)
- `PUT /clients/:id` (**protégé JWT**)
- `DELETE /clients/:id` (**protégé JWT**)

Validation minimale :

- `nom` min 2
- `email` contient `@`
- `telephone` optionnel

Partie F – Ressource “reservations” (obligatoire)

F1) Endpoints

- `GET /reservations` (**protégé JWT**) : liste
- `GET /reservations/:id` (**protégé JWT**) : détail
- `POST /reservations` (**protégé JWT**) : créer une réservation

Payload attendu :

```
{
  "client_id": 1,
  "motorcycle_id": 2,
  "date_debut": "2026-04-01",
  "date_fin": "2026-04-03"
}
```

F2) Règles métier minimales

À la création :

- `date_fin >= date_debut`
- le client existe
- la moto existe
- la moto est disponible (`disponible = 1`)
- pas de réservation qui chevauche les dates sur la même moto

Si une règle échoue :

- renvoyer `400` (validation) ou `409` (conflit) selon le cas

Partie G – Gestion d’erreurs (obligatoire)

G1) Format d’erreur JSON

Vous devez renvoyer un format cohérent :

```
{ "error": "Validation error", "details": ["..."] }
```

G2) Codes HTTP attendus (exemples)

- `200` OK
- `201` Created
- `204` No Content (DELETE)
- `400` Validation error
- `401` Unauthorized (token absent/invalide)
- `404` Not found
- `409` Conflict (ex : réservation impossible / email déjà utilisé)
- `500` Internal server error

Partie H – README (obligatoire)

Votre `README.md` doit contenir :

- prérequis (Node, MySQL)
- installation (`npm install`)

- création de la base (comment exécuter `schema.sql` et `seed.sql`)
- commande de lancement (`npm start`)
- liste des endpoints (au moins les principaux)
- comment tester avec `curl` ou Postman (2 exemples minimum)

Bonus (facultatif)

- **Pagination** sur `GET /motorcycles` et `GET /clients` (`?page=&limit=`)
- **Filtrage** sur `GET /motorcycles` (`?marque=...&disponible=1`)
- Route `GET /health` qui teste la connexion MySQL
- Ajout de rôles (`admin` / `employee`) et autorisation par rôle
- Tests automatiques (Jest + Supertest)

Conseils (très importants)

- Commencez par **schema.sql + seed.sql** et vérifiez vos tables.
- Implémentez une ressource à la fois : `motorcycles` → `clients` → `reservations` .
- Faites un commit Git à chaque étape importante (si votre enseignant le demande).
- Testez chaque route au fur et à mesure (Postman/Thunder Client/curl).